
Reproducing ALDA: Zero-Shot Generalization of Vision-Based RL Without Data Augmentation

Oleg Zhukov¹

Abstract

We reimplement Associative Latent Disentanglement (ALDA), which gives Soft Actor-Critic a QLAE-style disentangled latent space whose per-dimension codebooks act as an associative memory, and test whether it generalizes zero-shot to visual distribution shifts without any data augmentation. We train on DeepMind Control *walker-walk* from pixels for 1M environment steps (2 seeds) and evaluate periodically on a color-shift environment and on the Distracting Control Suite (easy: dynamic DAVIS backgrounds, camera and color perturbations). The agent reaches a return of 778 on the training environment and retains 81% of it under color shift and 43% under Distracting CS, with both OOD curves still rising at the end of training. Reconstructions show that the latent model maps distracted frames back to the training appearance. However, the size of this associative correction does not predict the per-evaluation performance gap.

1. Introduction

Policies learned from pixels typically break when the visual appearance of the scene changes. The dominant fix is data augmentation (Yarats et al., 2021a; Hansen et al., 2021): expose the agent to approximations of the test-time variations during training. ALDA (Batra & Sukhatme, 2025) uses no augmentation. It learns a disentangled latent representation with latent quantization (Hsu et al., 2023) and interprets the quantized latent model as an associative (Hopfield-like) memory: at test time, an out-of-distribution (OOD) encoding is pulled to the nearest stored in-distribution value in every latent dimension, so that a change in a task-irrelevant factor (e.g., background) ideally only perturbs a few latents, which are then snapped back.

¹Independent. Correspondence to: Oleg Zhukov <zhukovl3q@gmail.com>.

Test-task report: reproduction of ALDA (Batra & Sukhatme, 2025), October 2026.

This report describes a from-scratch, single-file implementation of SAC+ALDA built on CleanRL’s SAC (Huang et al., 2022), trains it on DeepMind Control (Tassa et al., 2018), and evaluates zero-shot transfer to the Distracting Control Suite (Stone et al., 2021). We ask one research question: *does the associative latent model actually act on OOD observations, and does the size of its correction explain the performance drop?* (Section 6).

2. Method

SAC. The base learner is Soft Actor-Critic (Haarnoja et al., 2018) with a tanh-Gaussian actor, two critics with Polyak-averaged targets, and automatic entropy tuning.

Disentanglement (QLAE). An encoder f_θ maps a single RGB frame to a continuous vector $z_{cont} \in \mathbb{R}^{n_z}$. Each dimension j owns a scalar codebook $v_j \in \mathbb{R}^{|V|}$, and a decoder g_ϕ reconstructs the frame from the quantized latent. Strong weight decay on f_θ and g_ϕ , together with the per-dimension quantization, encourages disentanglement (Hsu et al., 2023).

Association. ALDA replaces QLAE’s hard arg min with a modern Hopfield retrieval (Ramsauer et al., 2020) per dimension:

$$z_{d,j} = \sum_k \text{softmax}(-\beta |z_{cont,j} - v_{jk}|) v_{jk}. \quad (1)$$

With large β this retrieves (almost) one stored value per dimension. The codebook is *not* pulled towards encoder outputs (no $\mathcal{L}_{quantize}$); it is optimized only by the task (critic) loss, so the values act as task-optimized memories the encoder must learn to map onto. The ALDA objective is

$$\mathcal{L}_{ALDA} = \|g_\phi(\tilde{z}) - o\|_2^2 + \|z_{cont} - \text{sg}(z_d)\|_2^2 + \lambda_\theta \|\theta\|^2 + \lambda_\phi \|\phi\|^2, \quad (2)$$

where $\tilde{z} = z_{cont} + \text{sg}(z_d - z_{cont})$ is the straight-through estimator.

Temporal information. Frame stacks of $k=3$ are folded into the batch, each frame is encoded and associated independently, and the k latent vectors are then combined by a

Algorithm 1 SAC+ALDA update (one gradient step)

Input: batch (o, a, r, o', d) , $o \in \mathbb{R}^{B \times 3k \times 64 \times 64}$
 $x \leftarrow \text{reshape } o \text{ to } Bk \text{ single frames}$
 $z_{cont} \leftarrow f_\theta(x)$; $z_d \leftarrow l_\psi(\text{sg}(z_{cont}))$ (Equation (1))
 $\mathcal{L}_{ALDA} \leftarrow \text{MSE}(g_\phi(z_{cont} + \text{sg}(z_d - z_{cont})), x) + \text{MSE}(z_{cont}, \text{sg}(z_d))$
 $h \leftarrow h_\gamma(\text{reshape } z_d \text{ to } B \times k \times n_z)$
 $y \leftarrow r + \gamma(1-d)(\min_i \bar{Q}_i(h', a') - \alpha \log \pi(a'|h'))$, $a' \sim \pi(\cdot|h')$
 $\mathcal{L}_Q \leftarrow \sum_i (Q_i(h, a) - y)^2$
 step $(\theta, \phi, \psi, \gamma, Q)$ on $\mathcal{L}_Q + \mathcal{L}_{ALDA}$ (AdamW)
if $t \bmod 2 = 0$ **then**
 update π and α on $\text{sg}(h)$; Polyak-update $\bar{Q}, \bar{h}_\gamma$
end if

small 1D CNN h_γ over time (Algorithm 1). This keeps the autoencoder operating on single images, which the original paper found necessary for disentanglement.

3. Implementation

The code is a single file (`alda.py`, ~570 lines) with YAML configs; all hyperparameters follow the paper’s appendix table (Table 1).

Networks. The encoder and decoder follow QLAE’s appendix with leaky ReLUs replaced by GELU, as in ALDA. *Encoder:* 4 blocks of [conv3×3 s1, conv3×3 s1, conv4×4 s2] with widths 32/64/128/256, each conv followed by GELU and InstanceNorm ($64^2 \rightarrow 4^2$), then two dense-256 ReLU layers and a linear map to $n_z=12$. *Decoder:* StyleGAN-like; z goes through a 2-layer mapping MLP to a style vector w , a learned $256 \times 4 \times 4$ input is upsampled by 12 transposed-conv layers whose outputs are modulated by AdaIN from w , and a 1×1 conv outputs RGB. *Latent model:* 12×12 scalar codebooks, initialized to $\text{linspace}(-1, 1)$, $\beta=100$. *History encoder:* two Conv1d layers (kernel 2, 64 channels, GELU) over the 3 per-frame latents. The critic applies a linear projection to 50 features followed by two Q-MLPs (2×1024 , GELU); the actor has its own 50-d projection and a 2×1024 tanh-Gaussian MLP.

Gradient routing. We follow our reading of the paper’s Fig. 2. $\mathcal{L}_Q$ trains the Q-heads, the history encoder and the codebook values, but stops at z_{cont} : the encoder receives gradients *only* from reconstruction and commitment. The actor uses the critic’s history encoder output with a stop-gradient, as in SAC+AE (Yarats et al., 2021b).

Deviations and engineering choices. (i) **Decoupled weight decay.** With Adam and coupled L2 of 0.1, the weight-decay gradient swamped the reconstruction gradient and collapsed the encoder to a constant output early in

Table 1. Hyperparameters (identical for both seeds).

Parameter	Value
Replay capacity / batch size	10^6 / 128
Latent temperature β	100
Latents n_z / values per latent $ V $	12 / 12
Weight decay $\lambda_\theta = \lambda_\phi$ (AdamW)	0.1
Frame stack / action repeat	3 / 4
Observation	$9 \times 64 \times 64$
Learning rate (all nets) / α lr	10^{-3} / 10^{-4}
Initial temperature α	0.1
Actor / target update period	2 / 2
Polyak τ / discount γ	0.005 / 0.99
Feature dim (z_Q, z_π)	50
Random-action warmup	1000 agent steps

training. We therefore use AdamW (Loshchilov & Hutter, 2019), as in QLAE’s own implementation, with decay 0.1 on the encoder and decoder only. (ii) **Next-observation encoding.** Since frames are encoded independently, o' shares $k-1$ frames with o ; we reuse their latents and encode only the new frame for the TD target (exactly equivalent, $\sim 2 \times$ cheaper). (iii) **Efficiency.** The replay buffer stores only the newest frame per transition and rebuilds stacks at sampling time; the encoder/decoder run in bf16 autocast (latent model, critic and actor stay in fp32); the scene is rendered once per agent step instead of once per action repeat. (iv) **Time limits** are treated as truncation and do not cut the bootstrap.

4. Experimental Setup

Training. We train on *walker-walk* from 64×64 pixels for 10^6 environment steps (2.5×10^5 agent steps, one gradient update per agent step), with 2 seeds. Each run took roughly 9 hours on a single RTX 3090, including evaluation. No augmentation is used at any point; the agent only ever sees the standard DMC scene.

Zero-shot evaluation. Every 10^4 environment steps we run 5 deterministic episodes (tanh of the policy mean) in three environments, using seeds disjoint from training:

- **Train:** the unmodified DMC task.
- **Color shift:** the DCS color distraction with `max_delta=0.5` and no drift. Agent and floor RGB values are shifted by up to ± 0.5 once per episode. This is our stand-in for DMCG “color hard” used in the paper; it is similar in spirit but not identical.
- **Distracting CS (easy):** dynamic DAVIS 2017 (Pont-Tuset et al., 2017) validation videos as background (4 videos), plus dynamic camera and color perturbations at scale 0.1 (Stone et al., 2021).

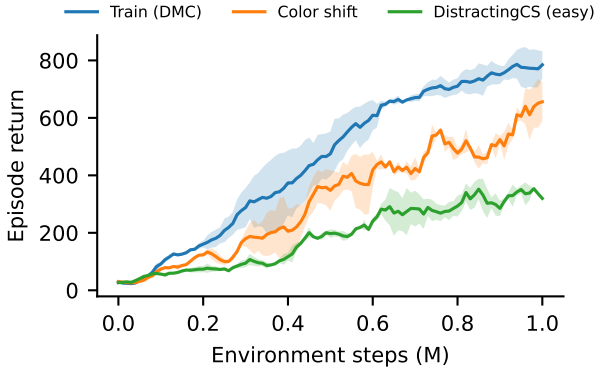


Figure 1. Evaluation return on walker-walk vs. environment steps. Lines are the mean over 2 seeds (moving average over 5 evaluations), shaded regions span the two seeds. The OOD environments are never seen during training.

Table 2. Evaluation return averaged over the last 10^5 steps (11 evaluations $\times$ 5 episodes). “Retained” is the ratio to the training-environment return.

Env	Seed 1	Seed 2	Mean	Retained
Train	722	833	778	1.00
Color shift	589	668	628	0.81
DistractingCS	329	339	334	0.43

Besides returns, at every evaluation we log latent statistics over all frames from the evaluation episodes: the mean associative shift $|z_{cont} - z_d|$, the fraction of z_{cont} entries outside their codebook’s range, the normalized entropy of codebook usage, and the reconstruction MSE against the observed frame.

5. Results

Learning curves. Figure 1 and Table 2 summarize performance. On the training environment both seeds reach 700–850 return by 1M steps, and the curve is still rising. Zero-shot performance appears in both OOD environments and keeps growing until the end of training: the color-shift return climbs to ~ 630 (81% retained) and Distracting CS to ~ 330 (43% retained), while a randomly initialized agent scores ~ 30 in all three. Variance is high in the OOD environments (per-evaluation standard deviation across 5 episodes of 150–200 vs. 40–50 on train): some episodes transfer almost perfectly while others fail, which fits the per-episode sampling of distractors.

Retention. Dividing the OOD return by the training return (Figure 2, left) removes the effect of the agent getting better at the task itself. Distracting CS retention is roughly flat

at 0.35–0.5 throughout training, so the absolute DCS gains in Figure 1 mostly track improvement on the training task. Color-shift retention dips to ~ 0.6 between 0.3M and 0.8M steps and recovers to ~ 0.85 by 1M steps. Codebook usage entropy (Figure 2, right) rises over the first 2×10^5 steps and then grows slowly. It is nearly identical on training and OOD frames, so OOD inputs are spread over the stored values as broadly as training inputs.

Reconstructions. Figure 3 shows the association step at work. Under color shift, the recolored walker (red, green, magenta) and tinted floors are decoded as the canonical orange walker on the blue floor, with the correct pose. Under Distracting CS, the DAVIS background (people, bicycles) is removed entirely; the decoder renders the training floor and sky. The errors are in the pose. Several DCS reconstructions show blurred or wrong limb configurations, likely because the camera perturbation moves the walker within the frame, which is a task-relevant change that the training data never covered.

Latent traversals. Figure 4 shows that individual latents control interpretable aspects of the walker’s configuration: some rows sweep a single leg forward/backward, rows 5 and 8 move the torso from upright to falling/lying, and others shift the body slightly in height or position. Because the training scene never changes appearance, no latent encodes background or color. This is expected, and it means OOD appearance can only be handled by association, since there is no distractor latent to absorb it. The latents are not perfectly disentangled: the torso latents also bend the legs. Codebook usage entropy at the end of training is 0.60 and 0.47 of the maximum (seed 1/2), i.e., most dimensions use only a subset of their 12 values.

6. Research Question: Does Association Explain Generalization?

ALDA’s central claim is that the latent model “remaps” OOD latents to in-distribution values. We ask two things: (a) is the associative correction measurably larger on OOD frames? (b) does its magnitude track how much return is lost?

(a) Association acts more on OOD frames. After the first 2×10^5 steps, the mean shift $|z_{cont} - z_d|$ is on average $1.73 \times$ larger on color-shift frames and $1.80 \times$ larger on DCS frames than on training frames (Figure 5, left). Early in training, while the encoder has not yet committed to the codebook, all three are high and the OOD shift is $\sim 2\text{--}3 \times$ the train shift. OOD encodings do *not* fall outside the codebook range more often (Figure 5, middle). On training frames $\sim 9\text{--}10\%$ of encoder outputs lie beyond the extreme codebook values (the encoder “overshoots” and is clipped

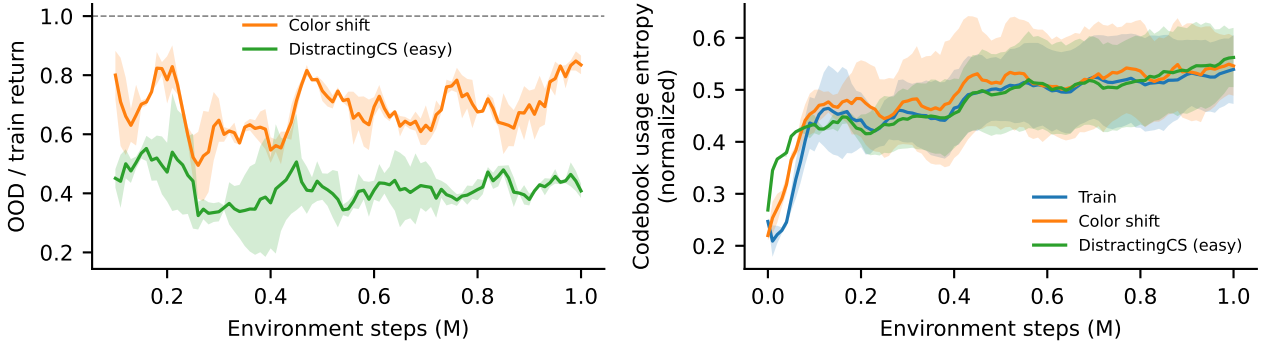


Figure 2. Left: OOD return as a fraction of the training-environment return (from 10^5 steps on, moving average over 5 evaluations; shaded: seed range). Right: normalized entropy of codebook usage on evaluation frames from each environment.



Figure 3. Inputs (top of each pair) and reconstructions $g_\phi(z_d)$ (bottom) at 1M steps, seed 1. From top: train, color shift, Distracting CS. OOD colors and DAVIS backgrounds are replaced by the training appearance (orange walker, blue checkered floor) while the pose is largely kept.



Figure 4. Latent traversals at 1M steps (seed 1). Row i sweeps latent i over its 12 sorted codebook values, with the others fixed at z_d of a training frame.

by association), but only $\sim 1\text{--}3\%$ do on DCS. OOD inputs therefore land *between* stored values, where a soft Hopfield retrieval with $\beta=100$ has to pick one of two neighbours and can pick the wrong one. The reconstruction error against the distracted frame (Figure 5, right) is $10\times$ (color) and $30\times$ (DCS) the training error and stays flat over training, consistent with the distractor being projected out instead of reconstructed (Figure 3).

(b) The size of the correction does not predict the loss. Pooling all evaluations after 2×10^5 steps (81 per seed, $n=162$), the Pearson correlation between the OOD-minus-train associative shift and the train-minus-OOD return gap is 0.20 for color shift and -0.24 for DCS. Both are weak, and

they have opposite signs. Over training the shift *decreases* (left panel) while the return gap *increases* in absolute terms (Figure 1). The average magnitude of association is a poor proxy for whether it helps. Figure 6 shows that what little correlation there is comes from training progress: early evaluations (dark) have a large shift and a small gap because returns are still low everywhere. Within the late, bright points there is no visible trend.

Answer. Yes, the associative latent model acts more strongly on OOD observations. Qualitatively it does what ALDA claims: distractor appearance is mapped back to the training appearance. The amount it moves the latents does

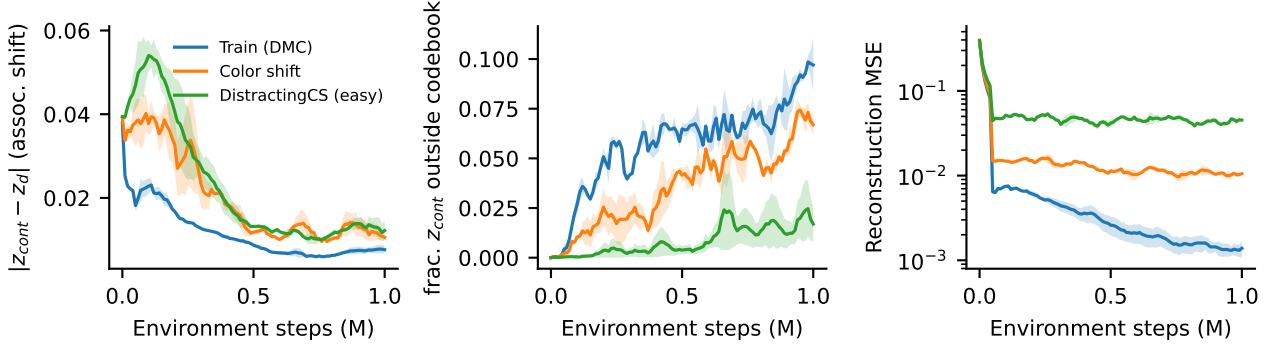


Figure 5. Latent statistics on evaluation frames over training (mean over 2 seeds, shaded: seed range). Left: mean associative shift $|z_{cont} - z_d|$. Middle: fraction of encoder outputs outside the range of their codebook. Right: reconstruction MSE of $g_\phi(z_d)$ against the observed (possibly distracted) frame, log scale.

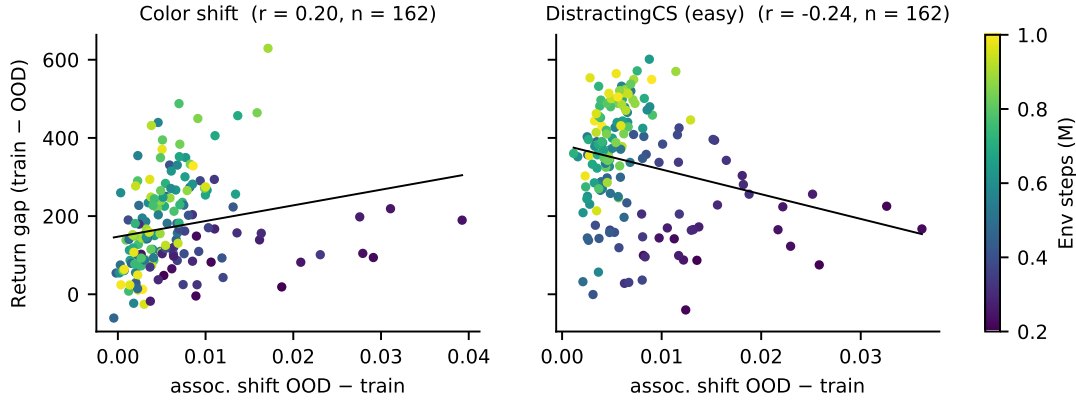


Figure 6. Per-evaluation associative shift (OOD minus train) vs. return gap (train minus OOD), both seeds, evaluations after 2×10^5 steps. Color encodes training progress; the line is a least-squares fit.

not predict performance. Our guess is that performance depends on whether the retrieved values keep the task-relevant factors (pose) intact: the DCS failures in Figure 3 are pose errors, and the background never leaks into the reconstruction. A more targeted diagnostic would measure retrieval errors in the latents that encode task-relevant factors, which requires ground-truth state labels for the evaluation frames. We leave this to future work.

7. Limitations and Open Items

- **Scope.** Only walker-walk with 2 seeds was trained to 1M steps. Configs for cartpole-balance, ball_in_cup-catch and finger-spin exist and pass short smoke tests, but were not run to completion. The paper uses 5 seeds and 4 tasks.
- **No baselines.** We did not train SAC+AE, SVEA or a SAC+QLAE ablation under the same budget, so we

cannot attribute the OOD returns to association specifically. The ablation that removes Equation (1) (hard quantization) is the most informative next experiment.

- **Environments.** Our color shift is the DCS color wrapper, not the DMCGB “color hard” environment, and we evaluate DCS only at the easy level.
- **Ambiguities.** The paper’s table lists Adam; we use AdamW for the encoder/decoder (Section 3). The exact gradient routing and the history-encoder architecture are inferred from the paper’s Fig. 2 and may differ from the authors’ code.

8. Conclusion

Our single-file SAC+ALDA learns walker-walk from pixels and, without any augmentation, transfers zero-shot to unseen color shifts (81% of training return) and partly to the Distracting Control Suite (43%). Both OOD curves

keep improving with training. Reconstructions show that the associative latent model maps distracted observations back to the training appearance. Its average correction magnitude, however, does not explain the residual performance gap, which appears to come from task-relevant (pose) errors under camera shift.

Software and Data

Code, configs and the plotting script for this report are available at <https://github.com/zxclwnq/test-alda>. Training: `python alda.py configs/walker_walk.yaml seed=1`. Figures: `python report/plots.py` and `python report/plots-extra.py`.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Batra, S. and Sukhatme, G. S. Zero-shot generalization of vision-based RL without data augmentation. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 1861–1870, 2018.
- Hansen, N., Su, H., and Wang, X. Stabilizing deep Q-learning with ConvNets and vision transformers under data augmentation. In *Advances in Neural Information Processing Systems*, volume 34, pp. 3680–3693, 2021.
- Hsu, K., Dorrell, W., Whittington, J., Wu, J., and Finn, C. Disentanglement via latent quantization. In *Advances in Neural Information Processing Systems*, volume 36, pp. 45463–45488, 2023.
- Huang, S., Dossa, R. F. J., Ye, C., Braga, J., Chakraborty, D., Mehta, K., and Araújo, J. G. CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 23(274):1–18, 2022.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Pont-Tuset, J., Perazzi, F., Caelles, S., Arbeláez, P., Sorkine-Hornung, A., and Van Gool, L. The 2017 DAVIS challenge on video object segmentation. *arXiv preprint arXiv:1704.00675*, 2017.
- Ramsauer, H., Schäfl, B., Lehner, J., Seidl, P., Widrich, M., Adler, T., Gruber, L., Holzleitner, M., Pavlović, M., Sandve, G. K., et al. Hopfield networks is all you need. *arXiv preprint arXiv:2008.02217*, 2020.
- Stone, A., Ramirez, O., Konolige, K., and Jonschkowski, R. The distracting control suite – a challenging benchmark for reinforcement learning from pixels. *arXiv preprint arXiv:2101.02722*, 2021.
- Tassa, Y., Doron, Y., Muldal, A., Erez, T., Li, Y., de Las Casas, D., Budden, D., Abdolmaleki, A., Merel, J., Lefrancq, A., et al. DeepMind control suite. *arXiv preprint arXiv:1801.00690*, 2018.
- Yarats, D., Fergus, R., Lazaric, A., and Pinto, L. Mastering visual continuous control: Improved data-augmented reinforcement learning. *arXiv preprint arXiv:2107.09645*, 2021a.
- Yarats, D., Zhang, A., Kostrikov, I., Amos, B., Pineau, J., and Fergus, R. Improving sample efficiency in model-free reinforcement learning from images. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pp. 10674–10681, 2021b.

A. Seed 2 Visualizations



Figure 7. Seed 2 at 1M steps. Left: latent traversals (as in Figure 4). Right: reconstructions on train, color shift and Distracting CS (as in Figure 3).

B. Training Dynamics

Reconstruction and commitment losses drop by an order of magnitude within the first 5×10^4 steps and then keep decreasing slowly (Figure 8). Commitment stays two orders of magnitude below reconstruction, i.e., the encoder stays close to the codebook without an explicit $\mathcal{L}_{quantize}$. Mean Q grows smoothly and the critic loss never diverges.

C. Evolution of Reconstructions and Traversals

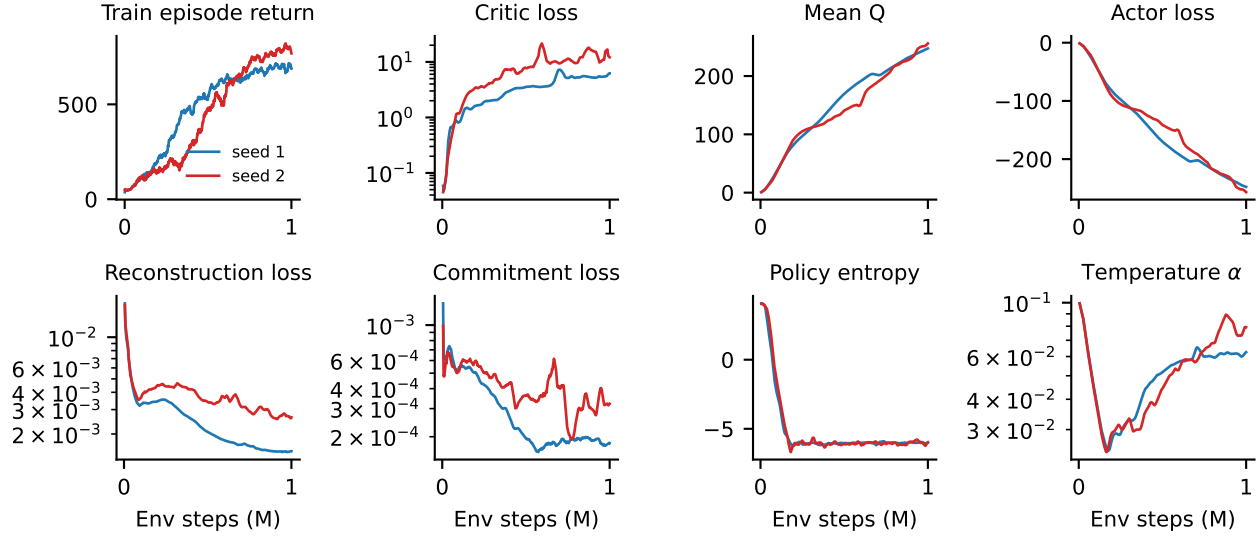


Figure 8. Per-episode training statistics (losses averaged over the updates within each episode, moving average over 20 episodes). Policy entropy reaches the SAC target $-|\mathcal{A}| = -6$ within $\sim 10^5$ steps, after which α is adjusted to hold it there. Seed 2 has a $\sim 2\times$ higher critic loss and a higher reconstruction/commitment loss, but also the higher return.



Figure 9. Inputs and reconstructions at 0, 0.1M, 0.3M, 0.6M and 1M steps (seed 1, from the TensorBoard image log). Left to right: train, color shift, Distracting CS. From 0.1M steps on, OOD backgrounds and colors are already decoded as the training scene; later training mainly sharpens the walker’s pose.

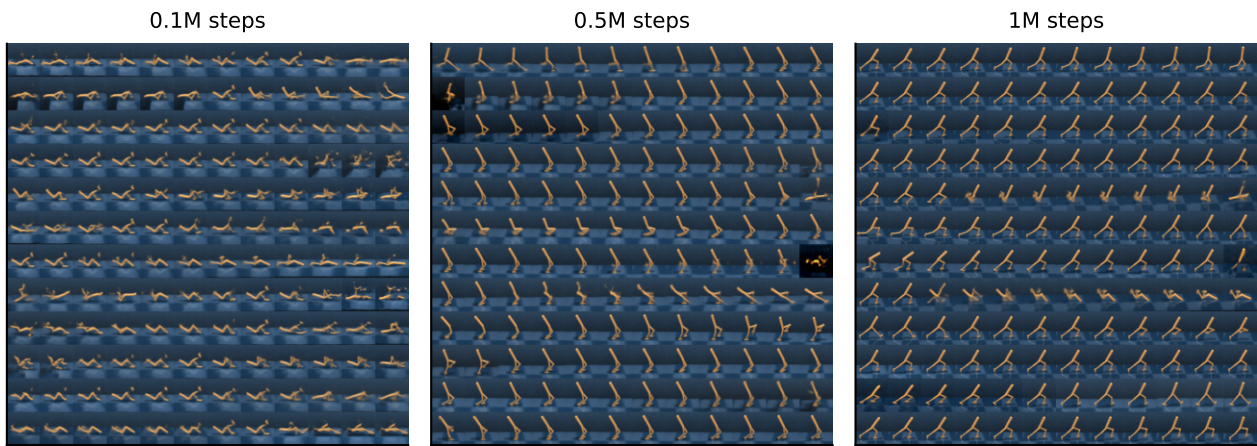


Figure 10. Latent traversals at 0.1M, 0.5M and 1M steps (seed 1). At 0.1M most latents barely change the image; by 0.5M several latents control distinct limb and torso configurations, and the traversals become smoother by 1M.